

#EVALUACIÓN PARCIAL N°3 - DEEP LEARNING(002D)

Integrantes:

- * Soledad Faúndez
- * Martin Osorio
- * Renato Espina
- * Esteban Olivares

Asignatura:

- * Deep Learning

Docente:

- * Marcelo Tapia Contreras

#Introducción

Un tipo de modelo RNN (Red Neuronal recurrente) es un modelo diseñado para poder procesar datos secuenciales como texto, voz o series temporales, el cual suele ser utilizado en la actualidad para poder generar traducciones, transcripciones de voz a texto, así como también poder predecir el clima, entre otros.

Por otro lado, un modelo LSTM es un modelo avanzado y mejorado del RNN, el cual fue diseñado con el propósito de cubrir la falta de memoria del modelo anterior y dar una solución a través de la selección de datos valiosos para poder recordarlos en las próximas secuencias.

Para el caso de este proyecto, se han de entrenar modelos que sean capaces de procesar una narración de literatura clásica — en este caso, Don Quijote de la Mancha — y a partir de esta poder generar narraciones nuevas, comparando sus resultados y desempeño a través de variadas métricas.

La narración obtenida de Don Quijote de la Mancha es un documento .txt que cuenta con la historia completa de Don Quijote de la Mancha, es decir, desde el primer capítulo hasta el capítulo 74, donde se es posible comprender la utilización de mayúsculas, minúsculas, puntos, comas, tildes entre otros.

#Carga de datos

```
In [ ]: #Se importan las librerias necesarias
import gdown
import numpy as np
import keras
import matplotlib.pyplot as plt
from keras.callbacks import LambdaCallback
from keras import layers
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LSTM
import random
import io
import re
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.metrics import precision_score, recall_score, f1_score,
classification_report

#Se descarga la data necesaria para poder trabajar el modelo, en este
caso, Don Quijote de la Mancha
file_id = '1sY9TTL-cMtKdZpVzIdYMiJqHBo2inW6D'
url = f'https://drive.google.com/uc?id={file_id}'
output = 'don_quijote.txt'

gdown.download(url, output, quiet=True)

'don_quijote.txt'
```

En primer lugar se hacen las importaciones necesarias para poder desarrollar los elementos necesarios a lo largo del cuaderno. En conjunto, se descarga el texto de Don Quijote de la Mancha para su procesamiento.

#Preprocesamiento de datos

##Visualización del texto

```
In [ ]: #Se visualiza una breve parte del contenido del texto
with open('don_quijote.txt', 'r', encoding='utf-8') as f:
    text = f.read()

print(text[:482])
```

Capítulo primero. Que trata de la condición y ejercicio del famoso hidalgo don Quijote de la Mancha

En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía un hidalgo de los de lanza en astillero, adarga antigua, rocín flaco y galgo corredor. Una olla de algo más vaca que carnero, salpicón las más noches, duelos y quebrantos los sábados, lantejas los viernes, algún palomino de añadidura los domingos, consumían las tres partes de su hacienda.

##Transformación a minúsculas

Tras ver una breve introducción de Don Quijote, transformamos las mayúsculas en minúsculas para poder facilitar la labor de entendimiento de nuestro modelo a generar.

```
In [ ]: #Se transforma el texto completo en minúsculas y se ve una breve parte
del contenido
text_lower = text.lower()
print(text_lower[:482])
```

capítulo primero. que trata de la condición y ejercicio del famoso hidalgo don quijote de la mancha

en un lugar de la mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía un hidalgo de los de lanza en astillero, adarga antigua, rocín flaco y galgo corredor. una olla de algo más vaca que carnero, salpicón las más noches, duelos y quebrantos los sábados, lantejas los viernes, algún palomino de añadidura los domingos, consumían las tres partes de su hacienda.

##Limpieza de texto

Con el propósito de seguir simplificando el aprendizaje de nuestro futuro modelo, se realizará una limpieza de caracteres que no serán considerados necesarios para la generación de texto. En este caso, se eliminarán los caracteres que no son considerados importantes a la hora de dar coherencia a una narrativa literaria. Ejemplos de estos caracteres pueden ser los números, comillas, saltos de línea extendidos, etc.

```
In [ ]: #Se eliminan números del texto
text_lower = re.sub(r'\d+', '', text_lower)

#Se eliminan caracteres especiales (Con excepciones)
text_lower = re.sub(r'^a-zAÉÍÓÚÑ\s.,;:!?¿¡\-'', text_lower)

#Se quitan saltos extendidos
text_lower = re.sub(r'\s+', ' ', text_lower).strip()

#Se verifican los resultados obtenidos
print(f"Carácteres únicos: {len(set(text_lower))}")
print(text_lower[:482])
```

```
Carácteres únicos: 42
capítulo primero. que trata de la condición y ejercicio del famoso
hidalgo don quijote de la mancha en un lugar de la mancha, de cuyo
nombre no quiero acordarme, no ha mucho tiempo que vivía un
hidalgo de los de lanza en astillero, adarga antigua, rocín flaco
y galgo corredor. una olla de algo más vaca que carnero, salpicón
las más noches, duelos y quebrantos los sábados, lantejas los
viernes, algún palomino de añadidura los domingos, consumían las
tres partes de su hacienda. e
```

##Mapeo de caracteres

Como ya se ha preparado el texto base para mantener caracteres importantes para la coherencia narrativa, a continuación se han de mapear para tener un diccionario de estos con índices únicos, esto permitirá "traducir" su contenido para que los modelos sean capaces de procesar los datos y poder generar información coherente y lógica.

```
In [ ]: vocabulary = sorted(set(text_lower))
vocab_size = len(vocabulary)

print(f"Caracteres únicos: {vocab_size}")
print(f"Vocabulario: {''.join(vocabulary)}")
```

```
Caracteres únicos: 42
Vocabulario: !,-.:;?abcdefghijklmnopqrstuvwxyz¡áéíñóúü
```

```
In [ ]: #Creación de dos diccionarios para caracteres y sus índices
char_to_index = {char: idx for idx, char in enumerate(vocabulary)}
index_to_char = {idx: char for idx, char in enumerate(vocabulary)}
```

##Conversión a secuencias (embedding)

A continuación, dividimos el texto en secuencias que poseen la misma longitud cada una. Esto se hace para no procesar todo de golpe.

```
In [ ]: #Se define la longitud de la secuencia
seq_length = 100
max_sequences = 500000

sequences = []
next_chars = []

for i in range(0, len(text_lower) - seq_length):
    sequences.append(text_lower[i: i + seq_length])
    next_chars.append(text_lower[i + seq_length])

print(f"Secuencias totales antes de filtrar: {len(sequences)}")

# LUEGO mezclar y limitar
perm = np.random.permutation(len(sequences))
sequences, next_chars = np.array(sequences), np.array(next_chars)
sequences, next_chars = sequences[perm], next_chars[perm]
sequences, next_chars = list(sequences[:max_sequences]),
list(next_chars[:max_sequences])

print(f"Secuencias después de filtrar: {len(sequences)}")
```

```
Secuencias totales antes de filtrar: 2061246
Secuencias después de filtrar: 500000
```

```
In [ ]: #Se transforman esas secuencias en números
X = np.zeros((len(sequences), seq_length, vocab_size), dtype=bool)
y = np.zeros((len(sequences), vocab_size), dtype=bool)

for i, sequence in enumerate(sequences):
    for t, char in enumerate(sequence):
        X[i, t, char_to_index[char]] = 1
    y[i, char_to_index[next_chars[i]]] = 1
```

```
In [ ]: #Tamaño del training set
print(f"Cantidad de secuencias, tamaño de las secuencias y tamaño del
vocabulario: ", {X.shape})
```

```
Cantidad de secuencias, tamaño de las secuencias y tamaño del
vocabulario: {(500000, 100, 42)}
```

```
In [ ]: #Split de train y validación
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.3,
    random_state=30
)

#Se divide el resto en val (15%) y test (15%)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.5,
    random_state=30
)

print(f"Train:      {X_train.shape}")
print(f"Validación: {X_val.shape}")
print(f"Test:       {X_test.shape}")
```

```
Train:      (350000, 100, 42)
Validación: (75000, 100, 42)
Test:       (75000, 100, 42)
```

#Definición del modelo

Luego de preprocesar los datos del texto de Don Quijote, se dará paso al trabajo de definición del modelo seleccionado para este cuaderno. En este caso, se decidió implementar un modelo de arquitectura LSTM con las siguientes características definidas a continuación.

```
In [ ]: #Función para crear el modelo con hiperparámetros configurables
def crear_modelo(units_1=256, units_2=64, dropout=0.2,
learning_rate=0.001):
    modelo = keras.Sequential([
        layers.LSTM(units=units_1, return_sequences=True, input_shape=
(seq_length, vocab_size)),
        layers.Dropout(dropout),
        layers.LSTM(units=units_2),
        layers.Dropout(dropout),
        layers.Dense(vocab_size, activation='softmax')
    ])
    modelo.compile(
        loss='categorical_crossentropy',
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        metrics=['accuracy']
    )
    return modelo
```

##Justificación de la selección del modelo y sus configuraciones

En este caso, para el problema anteriormente definido de generación de textos literarios a partir del libro Don Quijote de la Mancha, se decidió utilizar un modelo **LSTM** como arquitectura principal debido a las razones definidas a continuación:

- * Solución al problema de Vanishing Gradient: a pesar de que las RNN presentan cierta simplicidad a la hora de aplicarlas en comparación con LSTM, estas presentan dificultades debido al Vanishing Gradient. Al presentarse cierta longitud en la narración de Don Quijote, existe cierta coherencia o contexto que puede perderse si es que hay información que fue entregada con anterioridad en oraciones o párrafos pasados, por lo que la RNN puede tender a perder mucha información.

Podemos comprender que esta problemática anteriormente mencionada tiene solución al aplicarse un modelo de arquitectura LSTM, donde sus compuertas definen los siguientes procesos:

- * Compuerta de olvido: decide que información olvidar.
- * Compuerta de entrada: decide que información nueva guardar.
- * Compuerta de salida: decide que usar para la predicción.

Por otro lado, las configuraciones fueron definidas en base a las siguientes justificaciones:

- * Número de capas: se decidió utilizar dos capas para capturar distintos niveles de abstracción.
- * Unit 1: se utilizaron 256 units con el propósito de que el modelo aprenda del vocabulario, en conjunto con el tipo de escritura utilizado en Don Quijote.
- * Unit 2: con 64 units se reduce la dimensionalidad, evitando cuellos de botella.
- * Dropout: comprendiendo la dimensionalidad del texto, fue incluido para definir una base sin overfitting.
- * Learning rate: elección de parámetro base.
- * Adam: convergencia rápida y estable para utilizar como base.

#Entrenamiento y ajuste de hiperparámetros

A partir de la arquitectura seleccionada con anterioridad se decidió aplicar variadas configuraciones de hiperparámetros, esto con el propósito de identificar cuales funcionan mejor sobre los datos.

##Primer entrenamiento

En el primer entrenamiento se decidió trabajar con unas configuraciones moderadas y básicas para una evaluación simple sobre el modelo, manteniendo pocos epochs pero de batch_size mayor para procesar datos de forma rápida y eficiente.


```
In [ ]: np.random.seed(42)
        tf.random.set_seed(42)

        # Configuración 1 - base
        modelo_test1 = crear_modelo()

        hist_test1 = modelo_test1.fit(
            X_train, y_train,
            batch_size=128,
            epochs=20,
            validation_data=(X_val, y_val)
        )
```

Epoch 1/20

1m2735/2735 [0m 32m] [0m 37m] [0m 37m]
 1m68s [0m 24ms/step - accuracy: 0.3208 - loss: 2.2747 -
 val_accuracy: 0.3950 - val_loss: 1.9641

Epoch 2/20

1m2735/2735 [0m 32m] [0m 37m] [0m 37m]
 1m66s [0m 24ms/step - accuracy: 0.3639 - loss: 2.0972 -
 val_accuracy: 0.3988 - val_loss: 1.9396

Epoch 3/20

1m2735/2735 [0m 32m] [0m 37m] [0m 37m]
 1m66s [0m 24ms/step - accuracy: 0.3935 - loss: 1.9615 -
 val_accuracy: 0.4295 - val_loss: 1.8430

Epoch 4/20

1m2735/2735 [0m 32m] [0m 37m] [0m 37m]
 1m66s [0m 24ms/step - accuracy: 0.4320 - loss: 1.8407 -
 val_accuracy: 0.4716 - val_loss: 1.7167

Epoch 5/20

1m2735/2735 [0m 32m] [0m 37m] [0m 37m]
 1m67s [0m 25ms/step - accuracy: 0.4598 - loss: 1.7464 -
 val_accuracy: 0.4896 - val_loss: 1.6515

Epoch 6/20

1m2735/2735 [0m 32m] [0m 37m] [0m 37m]
 1m67s [0m 25ms/step - accuracy: 0.4792 - loss: 1.6836 -
 val_accuracy: 0.5079 - val_loss: 1.5915

Epoch 7/20

1m2735/2735 [0m 32m] [0m 37m] [0m 37m]
 1m67s [0m 25ms/step - accuracy: 0.4943 - loss: 1.6304 -
 val_accuracy: 0.5192 - val_loss: 1.5539

Epoch 8/20

1m2735/2735 [0m 32m] [0m 37m] [0m 37m]
 1m67s [0m 24ms/step - accuracy: 0.5044 - loss: 1.5974 -
 val_accuracy: 0.5262 - val_loss: 1.5299

Epoch 9/20

1m2735/2735 [0m 32m] [0m 37m] [0m 37m]
 1m69s [0m 25ms/step - accuracy: 0.5128 - loss: 1.5652 -
 val_accuracy: 0.5304 - val_loss: 1.5138

Epoch 10/20

1m2735/2735 [0m 32m] [0m 37m] [0m 37m]
 1m67s [0m 24ms/step - accuracy: 0.5185 - loss: 1.5470 -
 val_accuracy: 0.5344 - val_loss: 1.5000

Epoch 11/20

1m2735/2735 [0m 32m] [0m 37m] [0m 37m]
 1m68s [0m 25ms/step - accuracy: 0.5275 - loss: 1.5182 -

val_accuracy: 0.5397 - val_loss: 1.4840

Epoch 12/20

1m2735/2735 [0m 32m] [0m 37m] [0m

1m82s [0m 25ms/step - accuracy: 0.5277 - loss: 1.5175 -

val_accuracy: 0.5474 - val_loss: 1.4578

Epoch 13/20

1m2735/2735 [0m 32m] [0m 37m] [0m

1m66s [0m 24ms/step - accuracy: 0.5350 - loss: 1.4915 -

val_accuracy: 0.5517 - val_loss: 1.4376

Epoch 14/20

1m2735/2735 [0m 32m] [0m 37m] [0m

1m67s [0m 24ms/step - accuracy: 0.5411 - loss: 1.4669 -

val_accuracy: 0.5570 - val_loss: 1.4218

Epoch 15/20

1m2735/2735 [0m 32m] [0m 37m] [0m

1m67s [0m 25ms/step - accuracy: 0.5481 - loss: 1.4410 -

val_accuracy: 0.5603 - val_loss: 1.4090

Epoch 16/20

1m2735/2735 [0m 32m] [0m 37m] [0m

1m67s [0m 24ms/step - accuracy: 0.5539 - loss: 1.4220 -

val_accuracy: 0.5621 - val_loss: 1.4018

Epoch 17/20

1m2735/2735 [0m 32m] [0m 37m] [0m

1m67s [0m 25ms/step - accuracy: 0.5591 - loss: 1.4024 -

val_accuracy: 0.5645 - val_loss: 1.3956

Epoch 18/20

1m2735/2735 [0m 32m] [0m 37m] [0m

1m67s [0m 24ms/step - accuracy: 0.5634 - loss: 1.3856 -

val_accuracy: 0.5667 - val_loss: 1.3893

Epoch 19/20

1m2735/2735 [0m 32m] [0m 37m] [0m

1m67s [0m 25ms/step - accuracy: 0.5664 - loss: 1.3708 -

val_accuracy: 0.5687 - val_loss: 1.3837

Epoch 20/20

1m2735/2735 [0m 32m] [0m 37m] [0m

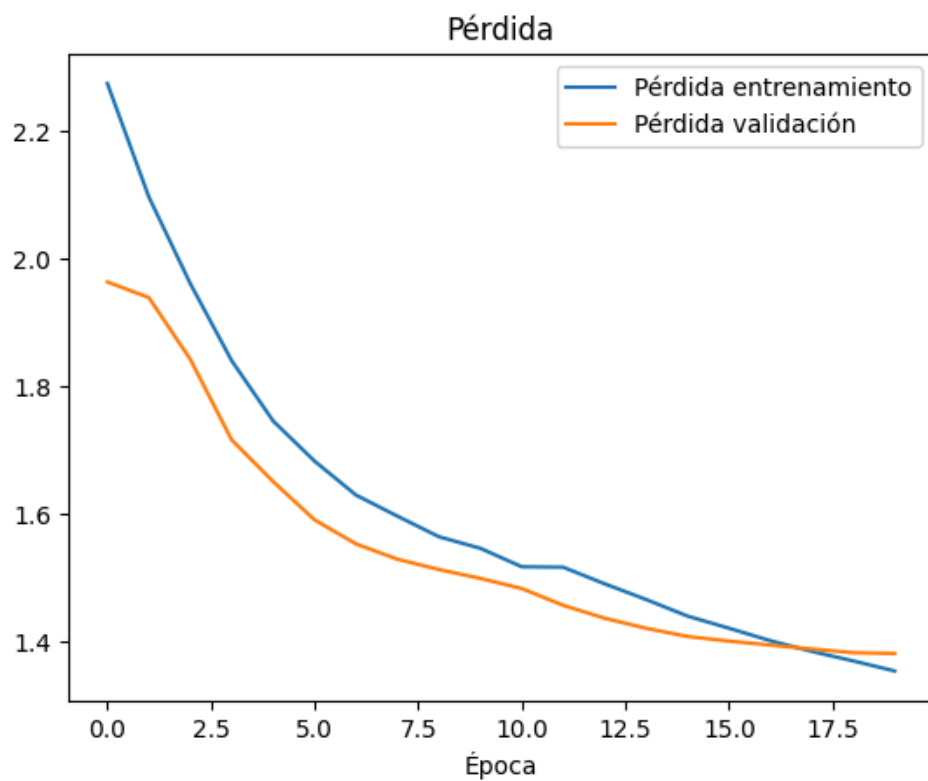
1m67s [0m 24ms/step - accuracy: 0.5703 - loss: 1.3550 -

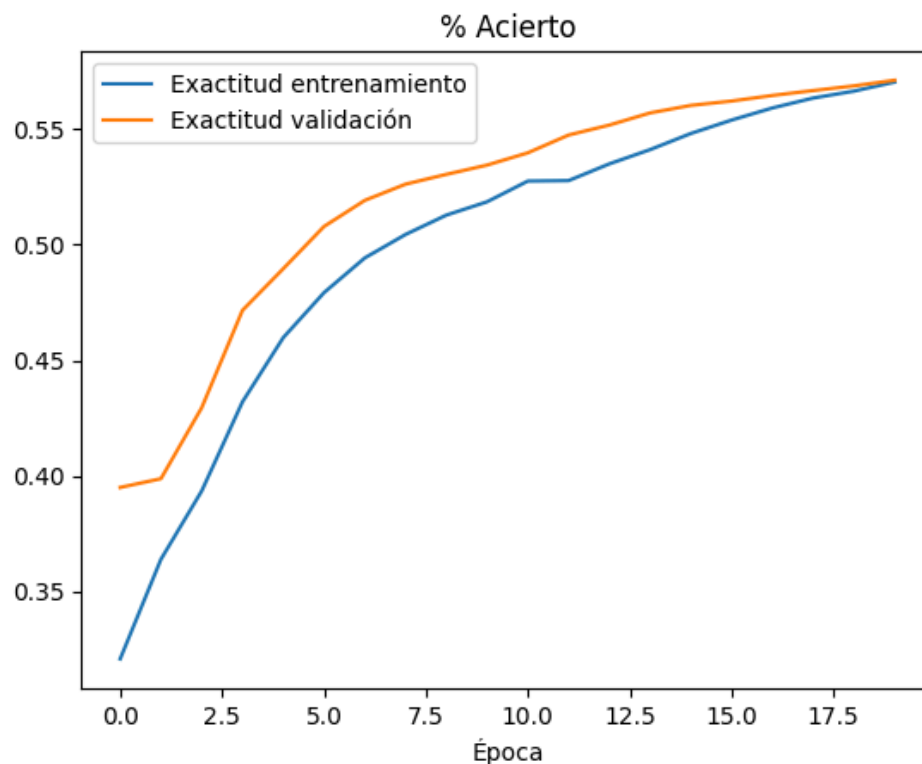
val_accuracy: 0.5711 - val_loss: 1.3824

```
In [ ]: # Mostramos la pérdida y el acierto por cada época

plt.plot(hist_test1.history['loss'], label='Pérdida entrenamiento')
plt.plot(hist_test1.history['val_loss'], label='Pérdida validación')
plt.legend()
plt.title('Pérdida')
plt.xlabel('Época')
plt.show()

plt.plot(hist_test1.history['accuracy'], label='Exactitud
entrenamiento')
plt.plot(hist_test1.history['val_accuracy'], label='Exactitud
validación')
plt.legend()
plt.title('% Acierto')
plt.xlabel('Época')
plt.show()
```





Perdida: Se comprenden buenos niveles de pérdida sobre el modelo base, los cuales se mantienen en niveles tanto en el entrenamiento como en la validación. Se demuestra de forma breve como es que la perdida en la validación tiende a ser ligeramente mejor que la de entrenamiento.

Aciertos: Ambas líneas suben de forma constante hasta mantenerse ligeramente aplanadas en la época 20, con un valor cercano al 0.55. Estos valores pueden sugerir que el modelo puede seguir siendo mejorado.

##Segundo entrenamiento

Como segunda configuración, se decidieron aumentar los valores para identificar si es que el modelo logra aprender más sobre los datos. Con más epochs y un batch_size mayor, la configuración busca comprender si es que este aumento realiza cambios significativos sobre el modelo.

```
In [ ]: np.random.seed(42)
        tf.random.set_seed(42)

# Configuración 2 - base
modelo_test2 = crear_modelo()

hist_test2 = modelo_test2.fit(
    X_train, y_train,
    batch_size=256,
    epochs=50,
    validation_data=(X_val, y_val)
)
```

```

/usr/local/lib/python3.12/dist-
packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object
as the first layer in the model instead.
    super().__init__(**kwargs)

```

Epoch 1/50

```
[1m1368/1368[0m [32m-----[0m[37m[0m  
[1m58s[0m [0m 41ms/step - accuracy: 0.2972 - loss: 2.3829 -  
val_accuracy: 0.3723 - val_loss: 2.0535
```

Epoch 2/50

```

[1m1368/1368[0m [32m-----[0m[0m[37m[0m
[1m54s[0m [39ms/step - accuracy: 0.3804 - loss: 2.0119 -
val accuracy: 0.4203 - val loss: 1.8746

```

Epoch 3/50

```
[1m1368/1368[0m [32m-----[0m[0m[37m[0m  
[1m54s[0m 39ms/step - accuracy: 0.4203 - loss: 1.8803 -  
val accuracy: 0.4573 - val loss: 1.7615
```

Epoch 4/50

```
[1m1368/1368][0m [32m-----[0m[37m[0m  
[1m54s[0m] 39ms/step - accuracy: 0.4533 - loss: 1.7801 -  
val accuracy: 0.4865 - val loss: 1.6680
```

Epoch 5/50

```

[1m1368/1368[0m [32m-----[0m[0m[37m[0m
[1m54s[0m 40ms/step - accuracy: 0.4543 - loss: 1.7856 -
val accuracy: 0.4797 - val loss: 1.6864

```

Epoch 6/50

```

[1m1368/1368[0m [32m-----[0m[37m[0m
[1m55s[0m [0m 40ms/step - accuracy: 0.4644 - loss: 1.7392 -
val accuracy: 0.4832 - val loss: 1.6721

```

Epoch 7/50

```

[1m1368/1368[0m [32m-----[0m[0m[37m[0m
[1m56s[0m [0m 41ms/step - accuracy: 0.4704 - loss: 1.7130 -
val accuracy: 0.5022 - val loss: 1.6077

```

Epoch 8/50

```

[1m1368/1368[0m [32m-----[0m[0m[37m[0m
[1m54s[0m [0m 40ms/step - accuracy: 0.5081 - loss: 1.5853 -
val accuracy: 0.5350 - val loss: 1.4986

```

Epoch 9/50

```

1m1368/1368[0m [32m-----[0m[37m[0m
1m54s[0m 40ms/step - accuracy: 0.5220 - loss: 1.5399 -

```

```
val_accuracy: 0.5430 - val_loss: 1.4736
Epoch 10/50
[1m1368/1368][0m [32m-----[0m[0m[37m[0m
[1m55s[0m 40ms/step - accuracy: 0.5303 - loss: 1.5087 -
val_accuracy: 0.5486 - val_loss: 1.4523
Epoch 11/50
[1m1368/1368][0m [32m-----[0m[0m[37m[0m
[1m54s[0m 40ms/step - accuracy: 0.5376 - loss: 1.4838 -
val_accuracy: 0.5535 - val_loss: 1.4361
Epoch 12/50
[1m1368/1368][0m [32m-----[0m[0m[37m[0m
[1m55s[0m 40ms/step - accuracy: 0.5438 - loss: 1.4596 -
val_accuracy: 0.5581 - val_loss: 1.4225
Epoch 13/50
[1m1368/1368][0m [32m-----[0m[0m[37m[0m
[1m55s[0m 40ms/step - accuracy: 0.5501 - loss: 1.4389 -
val_accuracy: 0.5601 - val_loss: 1.4095
Epoch 14/50
[1m1368/1368][0m [32m-----[0m[0m[37m[0m
[1m55s[0m 40ms/step - accuracy: 0.5547 - loss: 1.4206 -
val_accuracy: 0.5649 - val_loss: 1.3984
Epoch 15/50
[1m1368/1368][0m [32m-----[0m[0m[37m[0m
[1m56s[0m 41ms/step - accuracy: 0.5590 - loss: 1.4036 -
val_accuracy: 0.5655 - val_loss: 1.3946
Epoch 16/50
[1m1368/1368][0m [32m-----[0m[0m[37m[0m
[1m55s[0m 40ms/step - accuracy: 0.5645 - loss: 1.3870 -
val_accuracy: 0.5672 - val_loss: 1.3886
Epoch 17/50
[1m1368/1368][0m [32m-----[0m[0m[37m[0m
[1m55s[0m 40ms/step - accuracy: 0.5671 - loss: 1.3730 -
val_accuracy: 0.5689 - val_loss: 1.3851
Epoch 18/50
[1m1368/1368][0m [32m-----[0m[0m[37m[0m
[1m55s[0m 40ms/step - accuracy: 0.5713 - loss: 1.3598 -
val_accuracy: 0.5705 - val_loss: 1.3811
Epoch 19/50
[1m1368/1368][0m [32m-----[0m[0m[37m[0m
[1m56s[0m 41ms/step - accuracy: 0.5737 - loss: 1.3477 -
val_accuracy: 0.5722 - val_loss: 1.3791
Epoch 20/50
[1m1368/1368][0m [32m-----[0m[0m[37m[0m
[1m55s[0m 40ms/step - accuracy: 0.5781 - loss: 1.3353 -
val_accuracy: 0.5715 - val_loss: 1.3793
Epoch 21/50
[1m1368/1368][0m [32m-----[0m[0m[37m[0m
[1m55s[0m 40ms/step - accuracy: 0.5811 - loss: 1.3231 -
val_accuracy: 0.5718 - val_loss: 1.3815
Epoch 22/50
[1m1368/1368][0m [32m-----[0m[0m[37m[0m
[1m54s[0m 40ms/step - accuracy: 0.5839 - loss: 1.3150 -
val_accuracy: 0.5727 - val_loss: 1.3793
Epoch 23/50
[1m1368/1368][0m [32m-----[0m[0m[37m[0m
[1m55s[0m 40ms/step - accuracy: 0.5861 - loss: 1.3051 -
val_accuracy: 0.5736 - val_loss: 1.3807
```

Epoch 24/50

1m1368/1368 0m 32m 0m 37m 0m
1m55s 0m 40ms/step - accuracy: 0.5879 - loss: 1.2965 -
val_accuracy: 0.5729 - val_loss: 1.3828

Epoch 25/50

1m1368/1368 0m 32m 0m 37m 0m
1m55s 0m 40ms/step - accuracy: 0.5500 - loss: 1.4320 -
val_accuracy: 0.5250 - val_loss: 1.5364

Epoch 26/50

1m1368/1368 0m 32m 0m 37m 0m
1m55s 0m 40ms/step - accuracy: 0.5326 - loss: 1.4845 -
val_accuracy: 0.5326 - val_loss: 1.5118

Epoch 27/50

1m1368/1368 0m 32m 0m 37m 0m
1m55s 0m 40ms/step - accuracy: 0.5446 - loss: 1.4433 -
val_accuracy: 0.5729 - val_loss: 1.3848

Epoch 28/50

1m1368/1368 0m 32m 0m 37m 0m
1m55s 0m 40ms/step - accuracy: 0.5889 - loss: 1.2940 -
val_accuracy: 0.5461 - val_loss: 1.4696

Epoch 29/50

1m1368/1368 0m 32m 0m 37m 0m
1m55s 0m 40ms/step - accuracy: 0.5664 - loss: 1.3689 -
val_accuracy: 0.5555 - val_loss: 1.4412

Epoch 30/50

1m1368/1368 0m 32m 0m 37m 0m
1m55s 0m 40ms/step - accuracy: 0.5746 - loss: 1.3359 -
val_accuracy: 0.5745 - val_loss: 1.3842

Epoch 31/50

1m1368/1368 0m 32m 0m 37m 0m
1m54s 0m 40ms/step - accuracy: 0.5974 - loss: 1.2632 -
val_accuracy: 0.5743 - val_loss: 1.3874

Epoch 32/50

1m1368/1368 0m 32m 0m 37m 0m
1m55s 0m 40ms/step - accuracy: 0.5995 - loss: 1.2544 -
val_accuracy: 0.5761 - val_loss: 1.3901

Epoch 33/50

1m1368/1368 0m 32m 0m 37m 0m
1m55s 0m 40ms/step - accuracy: 0.6014 - loss: 1.2466 -
val_accuracy: 0.5751 - val_loss: 1.3931

Epoch 34/50

1m1368/1368 0m 32m 0m 37m 0m
1m55s 0m 40ms/step - accuracy: 0.6026 - loss: 1.2412 -
val_accuracy: 0.5735 - val_loss: 1.3988

Epoch 35/50

1m1368/1368 0m 32m 0m 37m 0m
1m55s 0m 40ms/step - accuracy: 0.6041 - loss: 1.2364 -
val_accuracy: 0.5739 - val_loss: 1.4029

Epoch 36/50

1m1368/1368 0m 32m 0m 37m 0m
1m55s 0m 40ms/step - accuracy: 0.6045 - loss: 1.2313 -
val_accuracy: 0.5731 - val_loss: 1.4023

Epoch 37/50

1m1368/1368 0m 32m 0m 37m 0m
1m55s 0m 40ms/step - accuracy: 0.6074 - loss: 1.2261 -
val_accuracy: 0.5735 - val_loss: 1.4008

Epoch 38/50

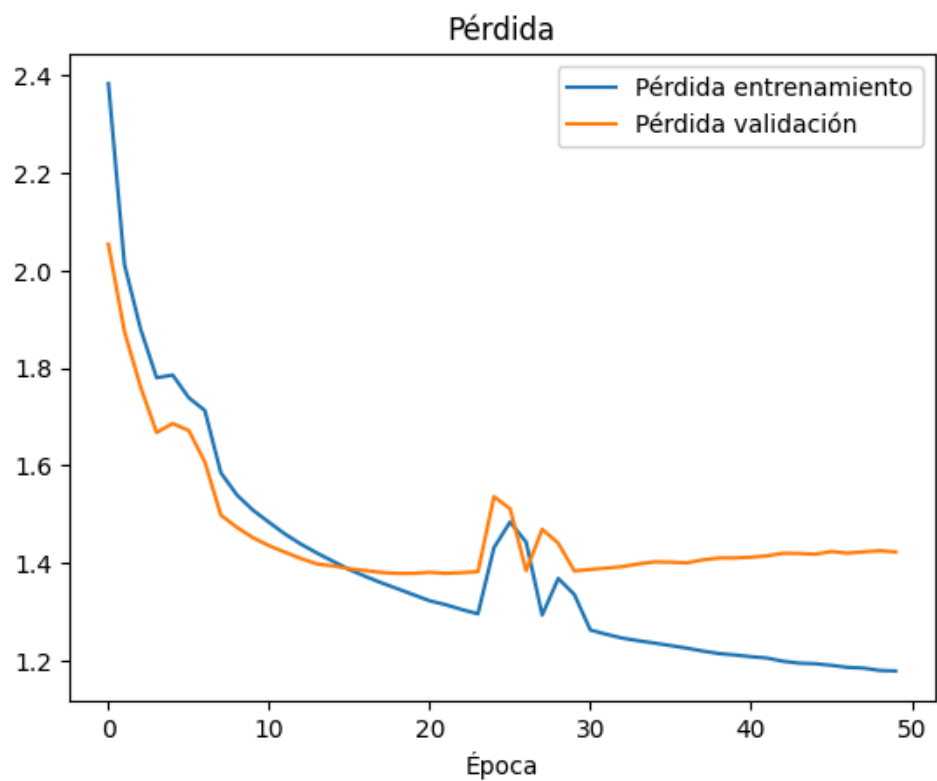
```
1m1368/1368 0m 32m 0m 37m 0m
1m54s 40ms/step - accuracy: 0.6075 - loss: 1.2201 -
val_accuracy: 0.5731 - val_loss: 1.4070
Epoch 39/50
1m1368/1368 0m 32m 0m 37m 0m
1m55s 40ms/step - accuracy: 0.6097 - loss: 1.2150 -
val_accuracy: 0.5722 - val_loss: 1.4107
Epoch 40/50
1m1368/1368 0m 32m 0m 37m 0m
1m55s 40ms/step - accuracy: 0.6108 - loss: 1.2121 -
val_accuracy: 0.5725 - val_loss: 1.4110
Epoch 41/50
1m1368/1368 0m 32m 0m 37m 0m
1m54s 40ms/step - accuracy: 0.6118 - loss: 1.2086 -
val_accuracy: 0.5724 - val_loss: 1.4123
Epoch 42/50
1m1368/1368 0m 32m 0m 37m 0m
1m83s 41ms/step - accuracy: 0.6124 - loss: 1.2058 -
val_accuracy: 0.5729 - val_loss: 1.4152
Epoch 43/50
1m1368/1368 0m 32m 0m 37m 0m
1m54s 39ms/step - accuracy: 0.6146 - loss: 1.1992 -
val_accuracy: 0.5722 - val_loss: 1.4205
Epoch 44/50
1m1368/1368 0m 32m 0m 37m 0m
1m54s 40ms/step - accuracy: 0.6144 - loss: 1.1953 -
val_accuracy: 0.5716 - val_loss: 1.4201
Epoch 45/50
1m1368/1368 0m 32m 0m 37m 0m
1m55s 40ms/step - accuracy: 0.6154 - loss: 1.1942 -
val_accuracy: 0.5714 - val_loss: 1.4186
Epoch 46/50
1m1368/1368 0m 32m 0m 37m 0m
1m54s 40ms/step - accuracy: 0.6159 - loss: 1.1909 -
val_accuracy: 0.5714 - val_loss: 1.4241
Epoch 47/50
1m1368/1368 0m 32m 0m 37m 0m
1m55s 40ms/step - accuracy: 0.6171 - loss: 1.1867 -
val_accuracy: 0.5720 - val_loss: 1.4207
Epoch 48/50
1m1368/1368 0m 32m 0m 37m 0m
1m54s 40ms/step - accuracy: 0.6180 - loss: 1.1853 -
val_accuracy: 0.5714 - val_loss: 1.4234
Epoch 49/50
1m1368/1368 0m 32m 0m 37m 0m
1m55s 40ms/step - accuracy: 0.6193 - loss: 1.1804 -
val_accuracy: 0.5731 - val_loss: 1.4256
Epoch 50/50
1m1368/1368 0m 32m 0m 37m 0m
1m83s 41ms/step - accuracy: 0.6186 - loss: 1.1793 -
val_accuracy: 0.5724 - val_loss: 1.4232
```

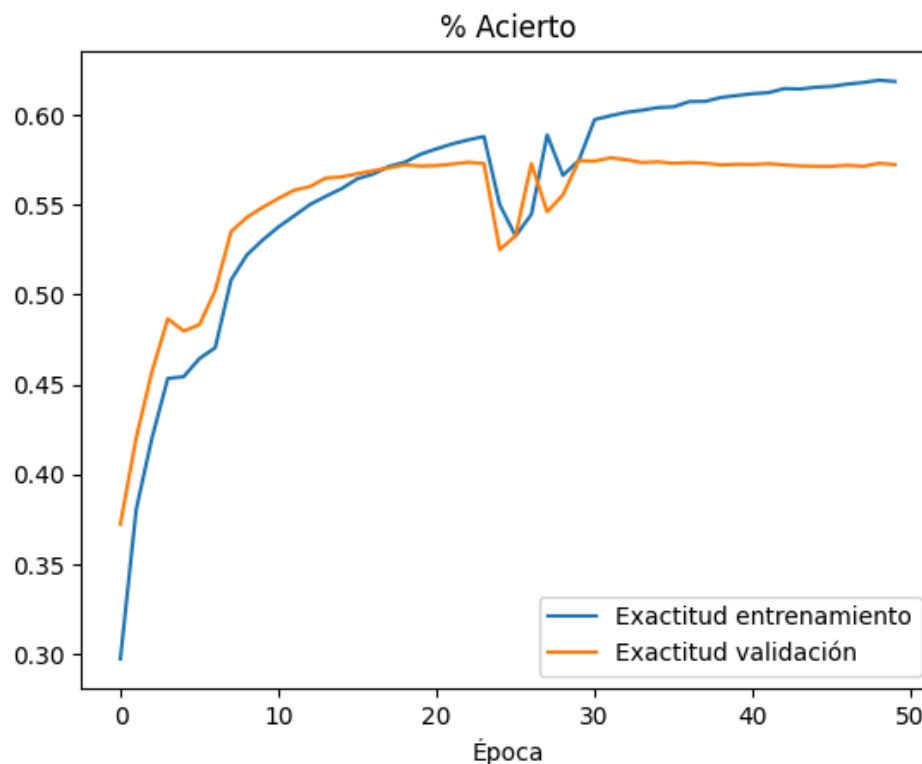


```
In [ ]: # Mostramos la pérdida y el acierto por cada época

plt.plot(hist_test2.history['loss'], label='Pérdida entrenamiento')
plt.plot(hist_test2.history['val_loss'], label='Pérdida validación')
plt.legend()
plt.title('Pérdida')
plt.xlabel('Época')
plt.show()

plt.plot(hist_test2.history['accuracy'], label='Exactitud
entrenamiento')
plt.plot(hist_test2.history['val_accuracy'], label='Exactitud
validación')
plt.legend()
plt.title('% Acierto')
plt.xlabel('Época')
plt.show()
```





Perdida: A pesar de mantener una perdida constante tanto en entrenamiento como en validación, se es posible ver como es que los valores suben u bajan de golpe, para luego visualizar como las líneas de entrenamiento y validación se dividen, demostrando que el modelo dejó de aprender a partir de los datos.

Acierto: de forma similar a la perdida, mantiene valores constantes tanto en entrenamiento como en la validación, manteniendose hasta que cerca de la época 25 estos bajan abruptamente y vuelven a aumentar, con la misma separación de líneas entre entrenamiento y validación. Esto es un claro signo de overfitting.

##Conclusiones parciales

* El incremento de hiperparámetros en el segundo entrenamiento reveló que alrededor de la epoca 25 existe cierta divergencia entre las líneas de entrenamiento y validación, en conjunto con una disminución abrupta de los valores. Este cambio puede deberse a una inadaptabilidad del learning rate en base al batch_size.

* En este caso, el aumento de los parámetros logró demostrar que estos no hacen una mejora en el modelo, por lo que se ha decidido **mantener un batch_size de 128**, en conjunto con una **disminución del learning rate** para evitar bajas de golpe en el entrenamiento. Por otro lado, también buscaremos aplicar un **aumento del dropout** para evitar depender de forma completa de ciertas neuronas, apoyando el aprendizaje costoso del modelo. Finalmente, se decidió **mantener las 50 épocas** para poder visualizar cambios de forma sostenida.

#Aplicación de cambios en el modelo

```
In [ ]: #Aplicación de las semillas
np.random.seed(42)
tf.random.set_seed(42)

#Se aplica la configuración 3 con las mejoras anteriormente mencionadas
modelo_test4 = crear_modelo(units_1=256, units_2=64, dropout=0.25,
learning_rate=0.0002)

hist_test4 = modelo_test4.fit(
    X_train, y_train,
    batch_size=128,
    epochs=50,
    validation_data=(X_val, y_val)
)
```

```
/usr/local/lib/python3.12/dist-
packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object
as the first layer in the model instead.
    super().__init__(**kwargs)
```

Epoch 1/50

```
1m2735/2735 [0m 32m ----- [0m 37m [0m
1m75s [0m 26ms/step - accuracy: 0.2504 - loss: 2.6491 -
val_accuracy: 0.3294 - val_loss: 2.2530
```

Epoch 2/50

```
1m2735/2735 [0m 32m ----- [0m 37m [0m
1m74s [0m 24ms/step - accuracy: 0.3287 - loss: 2.2212 -
val_accuracy: 0.3631 - val_loss: 2.1061
```

Epoch 3/50

```
1m2735/2735 [0m 32m ----- [0m 37m [0m
1m65s [0m 24ms/step - accuracy: 0.3554 - loss: 2.1145 -
val_accuracy: 0.3828 - val_loss: 2.0148
```

Epoch 4/50

```
1m2735/2735 [0m 32m ----- [0m 37m [0m
1m65s [0m 24ms/step - accuracy: 0.3744 - loss: 2.0429 -
val_accuracy: 0.4004 - val_loss: 1.9525
```

Epoch 5/50

```
1m2735/2735 [0m 32m ----- [0m 37m [0m
1m65s [0m 24ms/step - accuracy: 0.3907 - loss: 1.9869 -
val_accuracy: 0.4132 - val_loss: 1.9005
```

Epoch 6/50

```
1m2735/2735 [0m 32m ----- [0m 37m [0m
1m65s [0m 24ms/step - accuracy: 0.4049 - loss: 1.9385 -
val_accuracy: 0.4287 - val_loss: 1.8525
```

Epoch 7/50

```
1m2735/2735 [0m 32m ----- [0m 37m [0m
1m66s [0m 24ms/step - accuracy: 0.4167 - loss: 1.8992 -
val_accuracy: 0.4411 - val_loss: 1.8135
```

Epoch 8/50

```
1m2735/2735 [0m 32m ----- [0m 37m [0m
1m82s [0m 24ms/step - accuracy: 0.4266 - loss: 1.8654 -
val_accuracy: 0.4506 - val_loss: 1.7737
```

Epoch 9/50

1m2735/2735 [0m 32m [0m 37m 0m
1m65s [0m 24ms/step - accuracy: 0.4379 - loss: 1.8266 -
val_accuracy: 0.4615 - val_loss: 1.7428
Epoch 10/50
1m2735/2735 [0m 32m [0m 37m 0m
1m65s [0m 24ms/step - accuracy: 0.4477 - loss: 1.7950 -
val_accuracy: 0.4706 - val_loss: 1.7124
Epoch 11/50
1m2735/2735 [0m 32m [0m 37m 0m
1m65s [0m 24ms/step - accuracy: 0.4565 - loss: 1.7646 -
val_accuracy: 0.4802 - val_loss: 1.6818
Epoch 12/50
1m2735/2735 [0m 32m [0m 37m 0m
1m65s [0m 24ms/step - accuracy: 0.4649 - loss: 1.7386 -
val_accuracy: 0.4877 - val_loss: 1.6550
Epoch 13/50
1m2735/2735 [0m 32m [0m 37m 0m
1m65s [0m 24ms/step - accuracy: 0.4714 - loss: 1.7145 -
val_accuracy: 0.4941 - val_loss: 1.6342
Epoch 14/50
1m2735/2735 [0m 32m [0m 37m 0m
1m66s [0m 24ms/step - accuracy: 0.4786 - loss: 1.6895 -
val_accuracy: 0.4987 - val_loss: 1.6120
Epoch 15/50
1m2735/2735 [0m 32m [0m 37m 0m
1m65s [0m 24ms/step - accuracy: 0.4835 - loss: 1.6692 -
val_accuracy: 0.5030 - val_loss: 1.5934
Epoch 16/50
1m2735/2735 [0m 32m [0m 37m 0m
1m65s [0m 24ms/step - accuracy: 0.4902 - loss: 1.6487 -
val_accuracy: 0.5088 - val_loss: 1.5745
Epoch 17/50
1m2735/2735 [0m 32m [0m 37m 0m
1m65s [0m 24ms/step - accuracy: 0.4951 - loss: 1.6310 -
val_accuracy: 0.5122 - val_loss: 1.5606
Epoch 18/50
1m2735/2735 [0m 32m [0m 37m 0m
1m65s [0m 24ms/step - accuracy: 0.4997 - loss: 1.6136 -
val_accuracy: 0.5169 - val_loss: 1.5465
Epoch 19/50
1m2735/2735 [0m 32m [0m 37m 0m
1m65s [0m 24ms/step - accuracy: 0.5057 - loss: 1.5973 -
val_accuracy: 0.5201 - val_loss: 1.5359
Epoch 20/50
1m2735/2735 [0m 32m [0m 37m 0m
1m65s [0m 24ms/step - accuracy: 0.5086 - loss: 1.5830 -
val_accuracy: 0.5246 - val_loss: 1.5201
Epoch 21/50
1m2735/2735 [0m 32m [0m 37m 0m
1m65s [0m 24ms/step - accuracy: 0.5124 - loss: 1.5666 -
val_accuracy: 0.5278 - val_loss: 1.5105
Epoch 22/50
1m2735/2735 [0m 32m [0m 37m 0m
1m65s [0m 24ms/step - accuracy: 0.5167 - loss: 1.5550 -
val_accuracy: 0.5307 - val_loss: 1.4995
Epoch 23/50
1m2735/2735 [0m 32m [0m 37m 0m

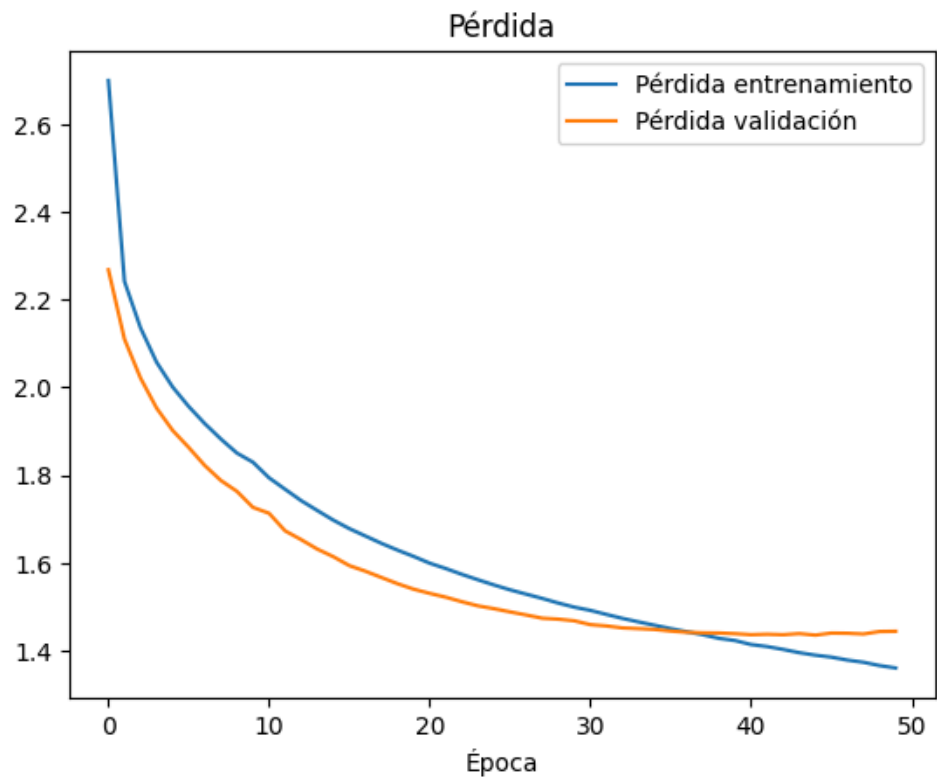
1m65s 0m 24ms/step - accuracy: 0.5209 - loss: 1.5430 -
val_accuracy: 0.5326 - val_loss: 1.4943
Epoch 24/50
1m2735/2735 0m 32m 0m 37m 0m
1m65s 0m 24ms/step - accuracy: 0.5242 - loss: 1.5297 -
val_accuracy: 0.5353 - val_loss: 1.4839
Epoch 25/50
1m2735/2735 0m 32m 0m 37m 0m
1m66s 0m 24ms/step - accuracy: 0.5268 - loss: 1.5198 -
val_accuracy: 0.5379 - val_loss: 1.4769
Epoch 26/50
1m2735/2735 0m 32m 0m 37m 0m
1m66s 0m 24ms/step - accuracy: 0.5297 - loss: 1.5096 -
val_accuracy: 0.5401 - val_loss: 1.4687
Epoch 27/50
1m2735/2735 0m 32m 0m 37m 0m
1m67s 0m 24ms/step - accuracy: 0.5332 - loss: 1.4991 -
val_accuracy: 0.5417 - val_loss: 1.4626
Epoch 28/50
1m2735/2735 0m 32m 0m 37m 0m
1m66s 0m 24ms/step - accuracy: 0.5362 - loss: 1.4886 -
val_accuracy: 0.5428 - val_loss: 1.4610
Epoch 29/50
1m2735/2735 0m 32m 0m 37m 0m
1m82s 0m 24ms/step - accuracy: 0.5394 - loss: 1.4790 -
val_accuracy: 0.5443 - val_loss: 1.4567
Epoch 30/50
1m2735/2735 0m 32m 0m 37m 0m
1m65s 0m 24ms/step - accuracy: 0.5415 - loss: 1.4688 -
val_accuracy: 0.5454 - val_loss: 1.4509
Epoch 31/50
1m2735/2735 0m 32m 0m 37m 0m
1m65s 0m 24ms/step - accuracy: 0.5440 - loss: 1.4603 -
val_accuracy: 0.5460 - val_loss: 1.4478
Epoch 32/50
1m2735/2735 0m 32m 0m 37m 0m
1m65s 0m 24ms/step - accuracy: 0.5453 - loss: 1.4525 -
val_accuracy: 0.5494 - val_loss: 1.4399
Epoch 33/50
1m2735/2735 0m 32m 0m 37m 0m
1m65s 0m 24ms/step - accuracy: 0.5481 - loss: 1.4443 -
val_accuracy: 0.5495 - val_loss: 1.4405
Epoch 34/50
1m2735/2735 0m 32m 0m 37m 0m
1m65s 0m 24ms/step - accuracy: 0.5508 - loss: 1.4369 -
val_accuracy: 0.5501 - val_loss: 1.4403
Epoch 35/50
1m2735/2735 0m 32m 0m 37m 0m
1m83s 0m 24ms/step - accuracy: 0.5532 - loss: 1.4270 -
val_accuracy: 0.5514 - val_loss: 1.4375
Epoch 36/50
1m2735/2735 0m 32m 0m 37m 0m
1m65s 0m 24ms/step - accuracy: 0.5550 - loss: 1.4178 -
val_accuracy: 0.5517 - val_loss: 1.4330
Epoch 37/50
1m2735/2735 0m 32m 0m 37m 0m
1m66s 0m 24ms/step - accuracy: 0.5573 - loss: 1.4114 -

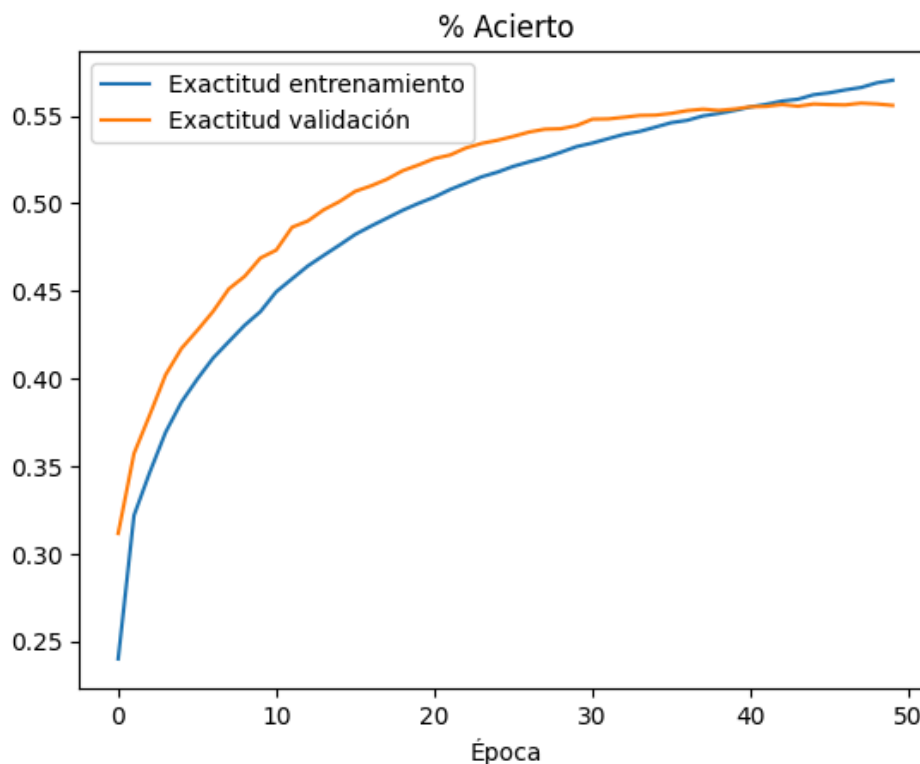
```
val_accuracy: 0.5529 - val_loss: 1.4323
Epoch 38/50
1m2735/2735 [0m 32m-----[0m37m[0m
1m65s [0m 24ms/step - accuracy: 0.5590 - loss: 1.4044 -
val_accuracy: 0.5546 - val_loss: 1.4288
Epoch 39/50
1m2735/2735 [0m 32m-----[0m37m[0m
1m66s [0m 24ms/step - accuracy: 0.5614 - loss: 1.3970 -
val_accuracy: 0.5536 - val_loss: 1.4316
Epoch 40/50
1m2735/2735 [0m 32m-----[0m37m[0m
1m65s [0m 24ms/step - accuracy: 0.5631 - loss: 1.3903 -
val_accuracy: 0.5555 - val_loss: 1.4301
Epoch 41/50
1m2735/2735 [0m 32m-----[0m37m[0m
1m65s [0m 24ms/step - accuracy: 0.5650 - loss: 1.3836 -
val_accuracy: 0.5553 - val_loss: 1.4320
Epoch 42/50
1m2735/2735 [0m 32m-----[0m37m[0m
1m65s [0m 24ms/step - accuracy: 0.5672 - loss: 1.3760 -
val_accuracy: 0.5553 - val_loss: 1.4320
Epoch 43/50
1m2735/2735 [0m 32m-----[0m37m[0m
1m66s [0m 24ms/step - accuracy: 0.5690 - loss: 1.3695 -
val_accuracy: 0.5561 - val_loss: 1.4312
Epoch 44/50
1m2735/2735 [0m 32m-----[0m37m[0m
1m66s [0m 24ms/step - accuracy: 0.5710 - loss: 1.3623 -
val_accuracy: 0.5543 - val_loss: 1.4343
Epoch 45/50
1m2735/2735 [0m 32m-----[0m37m[0m
1m66s [0m 24ms/step - accuracy: 0.5719 - loss: 1.3561 -
val_accuracy: 0.5562 - val_loss: 1.4351
Epoch 46/50
1m2735/2735 [0m 32m-----[0m37m[0m
1m82s [0m 24ms/step - accuracy: 0.5742 - loss: 1.3495 -
val_accuracy: 0.5545 - val_loss: 1.4361
Epoch 47/50
1m2735/2735 [0m 32m-----[0m37m[0m
1m66s [0m 24ms/step - accuracy: 0.5746 - loss: 1.3476 -
val_accuracy: 0.5543 - val_loss: 1.4412
Epoch 48/50
1m2735/2735 [0m 32m-----[0m37m[0m
1m65s [0m 24ms/step - accuracy: 0.5775 - loss: 1.3377 -
val_accuracy: 0.5555 - val_loss: 1.4396
Epoch 49/50
1m2735/2735 [0m 32m-----[0m37m[0m
1m65s [0m 24ms/step - accuracy: 0.5788 - loss: 1.3339 -
val_accuracy: 0.5567 - val_loss: 1.4388
Epoch 50/50
1m2735/2735 [0m 32m-----[0m37m[0m
1m65s [0m 24ms/step - accuracy: 0.5801 - loss: 1.3280 -
val_accuracy: 0.5557 - val_loss: 1.4440
```

```
In [ ]: # Mostramos la pérdida y el acierto por cada época

plt.plot(hist_test4.history['loss'], label='Pérdida entrenamiento')
plt.plot(hist_test4.history['val_loss'], label='Pérdida validación')
plt.legend()
plt.title('Pérdida')
plt.xlabel('Época')
plt.show()

plt.plot(hist_test4.history['accuracy'], label='Exactitud
entrenamiento')
plt.plot(hist_test4.history['val_accuracy'], label='Exactitud
validación')
plt.legend()
plt.title('% Acierto')
plt.xlabel('Época')
plt.show()
```





En este caso, al probar con diferentes hiperparámetros sobre el modelo, podemos comprender que el haber forzado un aprendizaje más costoso, en conjunto con el aumento del learning rate y el haber mantenido una cantidad alta de épocas no ha generado cambios significativos sobre el accuracy y la pérdida en base a lo que se ha construido en el primer modelo entrenado. Esto puede significar que no pueden realizarse más mejoras en base a los resultados del modelo, comprendiendo que al tener que predecir y generar el siguiente conjunto de palabras no se puede definir un contexto definitivo para indicar que existe una palabra correcta para seguir una narrativa dada.

Ejemplo de este caso sería una frase aleatoria como "cada uno es artifice de su propia -", donde puede darle continuación con variados ejemplos, como "ventura", "vida", "miseria", etc.

Debido a estos resultados previos, se continuará para dar paso a a lo que es la evaluación del modelo final.

#Evaluación del modelo

Para la evaluación del modelo seleccionado se aplicaron las métricas de accuracy, precision, recall y f1-score, con el propósito de comprender como es que está funcionando el modelo con el diccionario dado.


```

In [ ]: # Predicciones
y_pred = modelo_test4.predict(X_val)
y_pred_classes = np.argmax(y_pred, axis=1)
y_val_classes = np.argmax(y_val, axis=1)

# Etiquetas de los caracteres
labels = sorted(set(y_val_classes) | set(y_pred_classes))

def mostrar(c):
    if c == ' ':
        return "'.' (espacio)"
    return f"'{c}'"

target_names = [mostrar(index_to_char[i]) for i in labels]

print(classification_report(
    y_val_classes, y_pred_classes,
    labels=labels,
    target_names=target_names,
    zero_division=0
))

```

2/1m2344/2344 0m 2/32m — 2/0m 2/37m 0m
2/1m12s 0m 5ms/step

	precision	recall	f1-score	support
'.' (espacio)	0.74	0.90	0.81	13700
'!'	0.00	0.00	0.00	39
','	0.39	0.12	0.19	1489
'-'	0.49	0.65	0.56	248
'.'	0.67	0.02	0.04	283
':'	0.72	0.25	0.37	85
';'	0.00	0.00	0.00	170
'?'	0.00	0.00	0.00	25
'a'	0.52	0.59	0.55	6954
'b'	0.47	0.39	0.43	866
'c'	0.29	0.32	0.30	2085
'd'	0.32	0.53	0.40	3211
'e'	0.53	0.66	0.59	7854
'f'	1.00	0.00	0.01	261
'g'	0.48	0.23	0.31	677
'h'	0.65	0.32	0.43	689
'i'	0.58	0.43	0.49	2732
'j'	0.70	0.41	0.52	393
'l'	0.33	0.38	0.35	3195
'm'	0.34	0.16	0.22	1610
'n'	0.58	0.64	0.61	3877
'o'	0.64	0.59	0.61	5516
'p'	0.53	0.11	0.18	1291
'q'	0.43	0.23	0.30	1196
'r'	0.62	0.56	0.59	3673
's'	0.48	0.45	0.46	4506
't'	0.61	0.48	0.54	2163
'u'	0.76	0.59	0.66	2817
'v'	0.62	0.07	0.13	647
'x'	0.71	0.56	0.62	9

'y'	0.22	0.36	0.27	953
'z'	0.74	0.40	0.52	243
'i'	0.00	0.00	0.00	21
'¿'	0.00	0.00	0.00	42
'á'	0.47	0.11	0.17	262
'é'	0.53	0.11	0.19	228
'í'	0.64	0.33	0.43	472
'ñ'	0.48	0.38	0.42	133
'ó'	0.68	0.38	0.49	333
'ú'	1.00	0.12	0.21	51
'ü'	0.00	0.00	0.00	1
accuracy			0.56	75000
macro avg	0.49	0.31	0.34	75000
weighted avg	0.56	0.56	0.54	75000

A partir de los resultados, podemos interpretar los siguientes elementos:

* Se comprende una gran diferencia de valores entre el **macro avg y el weighted avg** (49-31-34/56-56-54, respectivamente). Esto puede deberse a la diferencia de uso entre los caracteres del vocabulario español, en este caso, se puede denotar que existen caracteres mucho más usados que otros a la hora de generar textos (ejemplo de esta diferencia puede ser el gran uso de vocales, a diferencia de la letra "z".)

* Cuando se revisan las métricas de los caracteres de forma individual, se es posible comprender que el modelo tiene una gran capacidad de predicción por sobre los espacios debido a su gran uso en textos para separar palabras. Estos resultados se ven seguidos por un promedio de predicción de vocales y consonantes, las cuales son usadas de forma frecuente para generar las palabras. Finalmente, existen otros caracteres que no han sido usados con tanta frecuencia, que denotan que son muy situacionales y que el modelo sacrifica de las generaciones de texto comunes.

* El accuracy del modelo se define con un valor de 0.56, indicando que el modelo tiene una buena capacidad de predicción sobre los caracteres, tomando en cuenta los variados usos del lenguaje en el español y los miles de casos que pueden aplicarse a la hora de generar palabras para un texto.

#Visualización de resultados

```

In [ ]: #A partir del uso de la temperatura, se puede variar la capacidad de
        generación de texto del modelo
        #Es decir, mientras menor sea la temperatura, será menos creativo y más
        repetitivo
        #Si la temperatura es mayor, es más creativo, pero es más propenso a
        cometer errores
        def sample(preds, temperature=1.0):
            preds = np.asarray(preds).astype('float64')
            preds = np.log(preds + 1e-9) / temperature
            exp_preds = np.exp(preds)
            preds = exp_preds / np.sum(exp_preds)
            probas = np.random.multinomial(1, preds, 1)
            return np.argmax(probas)

        #Función para generar texto, caráctes a carácter
        def generar_texto(modelo, semilla, longitud=400, temperature=1.0):
            generado = semilla
            sentence = semilla

            for _ in range(longitud):
                x_pred = np.zeros((1, seq_length, vocab_size))
                for t, char in enumerate(sentence):
                    x_pred[0, t, char_to_index[char]] = 1.0

            #El modelo predice la distribución del siguiente carácter
            preds = modelo.predict(x_pred, verbose=0)[0]

            #Se muestrea el carácter y se traduce de inedx a carácter
            next_index = sample(preds, temperature)
            next_char = index_to_char[next_index]

            #El carácter es agregado al resto del texto
            generado += next_char
            sentence = sentence[1:] + next_char

            return generado

        #Se elige una semilla aleatoria
        np.random.seed(42)
        start_index = np.random.randint(0, len(text_lower) - seq_length - 1)
        semilla = text_lower[start_index: start_index + seq_length]

        print("=" * 70)
        print("SEMILLA INICIAL (texto real de Don Quijote):")
        print("=" * 70)
        print(f'"{semilla}"\n')

        #Se generan los textos con las temperaturas diferentes
        for temp in [0.2, 0.5, 1.0]:
            print("=" * 70)
            print(f"TEXTO GENERADO | temperatura = {temp}")
            print("=" * 70)
            texto = generar_texto(modelo_test4, semilla, longitud=400,
            temperature=temp)
            print(texto[seq_length:])
            print("\n")

```

```

=====
====
SEMILLA INICIAL (texto real de Don Quijote):
=====
====
" consigo. así que, señor mío, estas honras que vuestra merced
quiere darme por ser ministro y adher"

=====
====
TEXTO GENERADO | temperatura = 0.2
=====
====
esto de la mano por el caballero de la mano de la mano de mi
parecer, y el pasar llegarme a la menos, que el cuento de la parte
de la verdad de la mano de la caballería, y al caballero de la
parte de la caballería de la caballería, y al parecer de la mano
de la mano, y con su casa de las de la caballería de la mancha,
que en este la mano de la mano de la caballería de la caballería
de la mancha, y

=====
====
TEXTO GENERADO | temperatura = 0.5
=====
====
es, muchos cortestos y endrandoses y a su suerte por parecerles de
la contancia, y le peres, que por las malas y por su parte de la
gamas, y allí le sucedía a los trastos de mi principa. y en el
agra entrado nuestro hermona de nadua, y hallaron de la señora,
andan el mundo el cielo, su puerdo y cantante en los voyos de la
cual quiere serced, mandares, porque en su ventano que se dio
desta vuestro

=====
====
TEXTO GENERADO | temperatura = 1.0
=====
====
ando, por descodia y de soro aldar bran por un pribente que se
estaban despago, no le cuerdao teniro en las lomas? el cielo, asi
pudiese hablando puentes se requitos. -dijo: -¡regato -respondió
don quijote-, pues si samo a honersa? todo es pañabra vigura como
dios como yo no, se nuvir desperalle alsí bueno pera ¿es que
estallo conocho le ha lloró. -y aindó a todos los hámones, como
dio protela co

```

#Conclusiones

- * El modelo fue capaz de alcanzar un accuracy que ronda alrededor de un 55-56%, con una pérdida del 1.4% demostrando así una buena capacidad de predicción por sobre los caracteres del alfabeto español.
- * Los resultados visuales del modelo fueron capaces de demostrar con certeza que este puede identificar letras para formar palabras coherentes del lenguaje, donde las temperaturas variaron su "creatividad", pero a su vez demostrando que también es propenso a cometer errores y formar palabras sin sentido.
- * Por otro lado, a través de las métricas pudimos identificar que elementos o caracteres son más propensos a ser utilizados por el modelo debido a su capacidad de identificación por sobre estos. Entre estos elementos destacan los espaciados entre palabra y palabra y vocales en conjunto con consonantes.
- * Se recomienda trabajar con un Transformer por sobre los datos con la intención de verificar si es que se es posible mejorar los resultados del modelo.

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).